

Testing considerations for sustainable workloads on Azure

Organizations that develop and deploy solutions to the cloud also need reliable testing. Learn about the considerations and recommendations for running workload tests and how to optimize for a more sustainable testing model.

Important

This article is part of the [Azure Well-Architected Framework sustainable workload](#) series. If you aren't familiar with this series, we recommend that you start with [What is a sustainable workload?](#)

Testing efficiency

Run intense testing during low-carbon periods

Running integration, performance, load, or other intensive testing capabilities might result in significant processing. A well-crafted testing design for deployed workloads can help ensure full utilization of available resources, which reduces carbon emissions.

Green Software Foundation alignment: [Carbon awareness](#)

Recommendation:

- Where energy mix data is available, plan to run testing when the datacenter primarily uses renewable energy. For example, in some regions, it might be more beneficial to run tests at night when cleaner energy sources are more prevalent.

Automate CI/CD to scale worker agents as needed

Running underused or inactive continuous integration and continuous delivery (CI/CD) agents results in more emissions.

Green Software Foundation alignment: [Hardware efficiency](#)

Recommendations:

- Keep the compute utilization high, based on the current demand, which avoids unnecessary capacity allocation.
- Scale out only when necessary and scale in when not testing. This approach ensures that there's no idle compute resources in test environments.
- Consider optimized platform services like containers over testing in a virtual machine (VM), which uses the platform to reduce maintenance.

Consider caching when you use CI/CD agents

Using caching mechanisms during CI/CD can reduce compute time, which reduces carbon emissions.

Green Software Foundation alignment: [Energy efficiency](#)

Recommendations:

- Store results from steps in a cache and reuse them between different CI/CD runs when possible. When a step requires CPU time to produce an artifact that doesn't often change between runs, save the artifact for future use. This optimization avoids wasting CPU time on every run that generates the same artifact repeatedly.
- Use a cache that's local to the CI/CD agent when it's self-hosted to further reduce data transfers and emissions. This setup ensures that the cache isn't transferred over the network, which can be a significant source of emissions.

Split large code repositories

Splitting large repositories can help the CI/CD phases where only the changed parts of the code are compiled. This strategy reduces compute time, which ultimately lowers carbon emissions.

Green Software Foundation alignment: [Energy efficiency](#)

Recommendations:

- Split large code repositories into smaller repositories and separate the main code from libraries and dependencies.
- Publish and reuse artifacts and libraries of code that are common across multiple repositories.

Profile and measure workloads

Measuring, profiling, and testing a workload is imperative to understanding how to best use allocated resources.

Assess where parallelization is possible

Without properly profiling and testing workloads, it's difficult to know whether they make the best use of the underlying platform and deployed resources.

Green Software Foundation alignment: [Measuring sustainability](#)

Recommendations:

- Test your applications to understand concurrent requests, simultaneous processing, and other factors.
- Consider GPU-based machines for better efficiency gains if you run machine learning for tests.
- Identify whether the workload is performance intensive and work toward optimization.
- **Consider the following trade-off:** Running GPU-based machines for machine learning tests might increase costs.

Assess by using chaos engineering

Running integration, performance, and load tests increases the reliability of a workload. However, the introduction of chaos engineering can significantly help improve reliability, resilience, and how the applications react to failures. You can optimize the workload to handle failures gracefully and waste fewer resources.

Green Software Foundation alignment: [Measuring sustainability](#)

Recommendations:

- Use load testing or [chaos engineering](#) to assess how the workload handles platform outages and traffic spikes or dips. This practice helps increase service resilience and the ability to react to failures, which allows for more optimized fault handling.
- Use chaos engineering to test energy faults or moments that emit more carbon. Consider setting up tests that challenge your application to consume as little energy as possible.

Define how the application should react to these conditions. Create a specific ecological version, also known as an *eco version*, that informs users they're emitting the minimum possible carbon by sacrificing some features and possibly some performance. This version can also serve as your benchmark application for scoring its sustainability.

- **Consider the following trade-off:** Injecting faults during chaos engineering and increasing the load on any system also increases the emissions used for the testing resources. Evaluate how and when you can use chaos engineering to increase workload reliability while considering the climate effects of running unnecessary testing sessions.

Establish CPU and memory thresholds in testing

Help build tests for testing sustainability in your application. Consider having a baseline CPU utilization measurement so that you can detect abnormal changes to the CPU utilization baseline when tests run. When you have a baseline, you can discover suboptimal decisions made in recent code changes earlier.

Adding tests and quality gates into the deployment and testing pipeline helps avoid deploying unsustainable solutions. This approach contributes to reduced emissions.

Green Software Foundation alignment: [Energy efficiency](#)

Recommendations:

- Monitor CPU and memory allocations when you run integration tests or unit tests.
- Find abnormally high resource consumption areas in the application code and focus on mitigating them first.
- Configure alerts or test failures if your application surpasses established baseline values. This configuration helps avoid deploying unsustainable workloads.
- **Consider the following trade-off:** As applications grow, the baseline might need to shift to avoid failing the tests when you introduce new features.

Related resource

- [Operational procedure considerations for sustainable workloads](#)

Last updated on 08/29/2025